

5장 4지선다형 30문항

I. 상속의 개념

1. 객체 지향에서 상속에 대한 설명으로 올바른 것은?

- ① 자식 클래스의 필드와 메소드를 부모 클래스가 물려받는 것이다.
- ② 부모 클래스의 필드와 메소드를 자식 클래스가 물려받는 것이다.
- ③ 서로 관계없는 두 클래스를 하나로 합치는 것이다.
- ④ 객체의 메모리를 자동으로 제거하는 것이다.

2. 객체 지향에서 상속을 사용하는 장점으로 적절하지 않은 것은?

- ① 클래스의 간결화
- ② 클래스 관리의 용이
- ③ 소프트웨어 생산성 향상
- ④ 동일한 멤버의 반복 작성 증가

3. 상속을 이용하면 자식 클래스가 간결해지는 주된 이유는?

- ① 모든 필드가 자동 삭제되기 때문에
- ② 동일한 멤버를 중복 작성할 필요가 없기 때문에
- ③ 생성자가 필요 없기 때문에
- ④ 객체를 생성하지 않아도 되기 때문에

4. 부모 클래스를 자바에서 무엇이라고 하는가?

- ① 서브 클래스
- ② 슈퍼 클래스
- ③ 내부 클래스
- ④ 객체 클래스

5. 자식 클래스를 자바에서 무엇이라고 하는가?

- ① 슈퍼 클래스
- ② 메인 클래스
- ③ 서브 클래스
- ④ 기본 클래스

상속은 부모 클래스의 필드와 메소드를 자식 클래스가 물려받는 것이며, 멤버의 중복 작성을 줄여 클래스를 간결하게 만들 수 있다.

II. 클래스 상속

6. 자바에서 클래스를 상속할 때 사용하는 키워드는?

- ① implements
- ② extends

③ inherits

④ super

7. 다음 중 Person 클래스를 상속받는 Student 클래스의 올바른 선언은?

① `class Student inherit Person`

② `class Student extends Person`

③ `class Person extends Student`

④ `extends Student class Person`

8. 자바의 클래스 상속에 대한 설명으로 올바른 것은?

① 클래스의 다중 상속을 지원한다.

② 상속은 한 번만 가능하다.

③ 클래스의 다중 상속을 지원하지 않는다.

④ 모든 클래스는 반드시 두 개의 클래스를 상속해야 한다.

9. 자바에서 상속 횟수에 대한 설명으로 올바른 것은?

① 한 번만 가능하다.

② 두 번까지만 가능하다.

③ 세 번까지만 가능하다.

④ 상속 횟수에는 제한이 없다.

10. 자바의 모든 클래스가 자동으로 상속받는 최상위 클래스는?

① `java.lang.String`

② `java.lang.System`

③ `java.lang.Object`

④ `java.lang.Class`

자바에서는 `extends` 로 상속을 선언하며, 클래스의 다중 상속은 지원하지 않지만 상속 횟수에는 제한이 없다. 모든 클래스의 최상위 조상은 `java.lang.Object` 이다.

III. 상속과 멤버 접근

11. 다음 코드에서 ColorPoint의 슈퍼 클래스는?

```
class Point {  
}  
  
class ColorPoint extends Point {  
}
```

① ColorPoint

② Point

③ Object만 가능

④ extends

12. 위 코드에서 ColorPoint 클래스가 Point의 `public` 메소드를 사용하는 것은?

- ① 불가능하다.
- ② 가능하다.
- ③ 생성자에서만 가능하다.
- ④ `main()`에서만 가능하다.

13. 슈퍼 클래스의 `private` 멤버에 대한 설명으로 올바른 것은?

- ① 모든 서브 클래스가 직접 접근할 수 있다.
- ② 같은 패키지라면 직접 접근할 수 있다.
- ③ 해당 클래스 내부에서만 직접 접근할 수 있다.
- ④ 모든 클래스에서 접근할 수 있다.

14. 슈퍼 클래스의 `public` 멤버에 대한 설명으로 올바른 것은?

- ① 해당 클래스에서만 접근 가능하다.
- ② 같은 패키지에서만 접근 가능하다.
- ③ 서브 클래스에서 접근할 수 없다.
- ④ 다른 모든 클래스에서 접근할 수 있다.

15. 슈퍼 클래스의 `protected` 멤버에 대한 설명으로 올바른 것은?

- ① 같은 패키지의 클래스에서 접근할 수 있다.
- ② 서브 클래스에서도 접근할 수 없다.
- ③ 슈퍼 클래스 내부에서만 접근할 수 있다.
- ④ 무조건 `private`과 동일하다.

16. 다른 패키지에 있는 서브 클래스가 슈퍼 클래스의 어떤 멤버에 접근할 수 있는가?

- ① `private` 멤버
- ② `protected` 멤버
- ③ 모든 `default` 멤버
- ④ 모든 `private` 멤버

17. 슈퍼 클래스의 `default` 멤버에 대한 설명으로 올바른 것은?

- ① 같은 패키지의 클래스에서 접근할 수 있다.
- ② 모든 패키지에서 접근할 수 있다.
- ③ 해당 클래스에서만 접근할 수 있다.
- ④ 어떠한 클래스에서도 접근할 수 없다.

18. 다음 코드에서 Student가 직접 접근할 수 없는 필드는?

```
class Person {  
    private int weight;  
    int age;  
    protected int height;  
    public String name;  
}
```

```

}

class Student extends Person {
}

```

- ① weight
- ② age
- ③ height
- ④ name

자료의 예제에서도 `private weight` 는 Student에서 직접 접근할 수 없으며 `setWeight()` 와 같은 공개 메소드를 통해 간접적으로 접근한다.

IV. 슈퍼 클래스와 서브 클래스의 생성자

19. 서브 클래스 객체가 생성될 때 실행되는 생성자는?

- ① 서브 클래스 생성자만 실행된다.
- ② 슈퍼 클래스 생성자만 실행된다.
- ③ 슈퍼 클래스와 서브 클래스 생성자가 모두 실행된다.
- ④ 어떠한 생성자도 실행되지 않는다.

20. 서브 클래스 객체 생성 시 생성자의 실제 실행 순서로 올바른 것은?

- ① 서브 클래스 생성자 → 슈퍼 클래스 생성자
- ② 슈퍼 클래스 생성자 → 서브 클래스 생성자
- ③ main() → 서브 클래스 생성자
- ④ 실행 순서는 항상 임의로 결정된다.

21. 다음 코드의 출력 순서는?

```

class A {
    A() {
        System.out.print("A ");
    }
}

class B extends A {
    B() {
        System.out.print("B ");
    }
}

B b = new B();

```

- ① B A
- ② A B
- ③ B
- ④ A

22. 다음과 같이 상속 관계가 연결되어 있을 때 `new C()` 를 실행한 경우 생성자의 실행 순서는?

```
class A { }  
class B extends A { }  
class C extends B { }
```

- ① C → B → A
- ② A → B → C
- ③ B → C → A
- ④ C만 실행

서브 클래스 객체를 생성하면 슈퍼 클래스와 서브 클래스 생성자가 모두 실행되며, 실제 실행은 슈퍼 클래스 생성자가 먼저 이루어진다.

V. super()와 생성자 선택

23. 서브 클래스에서 슈퍼 클래스의 생성자를 명시적으로 선택하여 호출할 때 사용하는 것은?

- ① `this()`
- ② `new()`
- ③ `super()`
- ④ `extends()`

24. 서브 클래스 생성자에서 슈퍼 클래스 생성자를 명시적으로 선택하지 않으면 어떻게 되는가?

- ① 아무 생성자도 호출되지 않는다.
- ② 컴파일러가 슈퍼 클래스의 기본 생성자를 자동 선택한다.
- ③ 서브 클래스의 기본 생성자가 두 번 실행된다.
- ④ 프로그램이 항상 종료된다.

25. 슈퍼 클래스에 기본 생성자가 없고 매개변수가 있는 생성자만 있는데, 서브 클래스 생성자에서 `super()` 를 작성하지 않았다. 결과는?

- ① 정상 실행된다.
- ② 슈퍼 클래스 생성자가 실행되지 않는다.
- ③ 컴파일 오류가 발생할 수 있다.
- ④ 서브 클래스 생성자가 자동 삭제된다.

26. `super(parameter);` 의 기능으로 올바른 것은?

- ① 서브 클래스의 생성자를 다시 호출한다.
- ② 인자에 맞는 슈퍼 클래스 생성자를 호출한다.
- ③ 객체를 삭제한다.
- ④ 슈퍼 클래스를 새로 선언한다.

27. `super()` 호출문은 서브 클래스 생성자의 어느 위치에 작성해야 하는가?

- ① 마지막 줄

- ② 첫 번째 줄
- ③ 어느 위치나 가능
- ④ main() 안

`super()` 는 슈퍼 클래스 생성자를 명시적으로 선택하는 데 사용하며, 반드시 서브 클래스 생성자의 첫 번째 문장에 위치해야 한다.

VI. 코드 이해

28. 다음 코드의 실행 결과는?

```
class A {
    A() {
        System.out.print("A ");
    }

    A(int x) {
        System.out.print("A" + x + " ");
    }
}

class B extends A {
    B(int x) {
        super(x);
        System.out.print("B" + x);
    }
}

B b = new B(5);
```

- ① B5 A5
- ② A B5
- ③ A5 B5
- ④ B5

29. 다음 코드에서 `super(x, y);` 가 호출하는 생성자는?

```
class Point {
    Point() { }

    Point(int x, int y) { }
}

class ColorPoint extends Point {
    ColorPoint(int x, int y, String color) {
        super(x, y);
    }
}
```

- ① `Point()`
- ② `Point(int x, int y)`
- ③ `ColorPoint()`

④ Object()

30. 다음 코드의 출력 결과는?

```
class Point {
    private int x, y;

    Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    void showPoint() {
        System.out.print("(" + x + "," + y + ")");
    }
}

class ColorPoint extends Point {
    private String color;

    ColorPoint(int x, int y, String color) {
        super(x, y);
        this.color = color;
    }

    void showColorPoint() {
        System.out.print(color);
        showPoint();
    }
}

ColorPoint cp = new ColorPoint(5, 6, "blue");
cp.showColorPoint();
```

① (5,6)blue

② blue(5,6)

③ blue 5 6

④ 컴파일 오류

답

문항	정답	문항	정답	문항	정답
1	②	11	②	21	②
2	④	12	②	22	②
3	②	13	③	23	③
4	②	14	④	24	②
5	③	15	①	25	③
6	②	16	②	26	②
7	②	17	①	27	②
8	③	18	①	28	③
9	④	19	③	29	②

문항	정답	문항	정답	문항	정답
10	③	20	②	30	②

5장 4지선다형 30문항 해설

문항	정답	해설
1	②	상속은 부모 클래스에 만들어진 필드와 메소드를 자식 클래스가 물려받는 것이다.
2	④	상속을 사용하면 멤버의 중복 작성이 줄어든다. 따라서 '동일한 멤버의 반복 작성 증가'는 상속의 장점이 아니다.
3	②	부모 클래스의 공통 멤버를 물려받으므로 자식 클래스에서 동일한 내용을 다시 작성할 필요가 없어 클래스가 간결해진다.
4	②	부모 클래스는 슈퍼 클래스(super class)라고 한다.
5	③	자식 클래스는 서브 클래스(sub class)라고 한다.
6	②	자바에서 클래스를 상속할 때는 <code>extends</code> 키워드를 사용한다.
7	②	<code>class Student extends Person</code> 은 Student가 Person을 상속한다는 의미이다.
8	③	자바는 클래스의 다중 상속을 지원하지 않는다.
9	④	자료에서는 자바의 상속 횟수는 무제한이라고 설명한다.
10	③	자바 상속 구조의 최상위 조상 클래스는 <code>java.lang.Object</code> 이며 모든 클래스가 자동으로 이를 상속받는다.

11~18. 상속과 멤버 접근

문항	정답	해설
11	②	<code>ColorPoint extends Point</code> 이므로 Point가 슈퍼 클래스이고 ColorPoint가 서브 클래스이다.
12	②	ColorPoint는 상속받은 Point의 <code>public</code> 메소드를 사용할 수 있다. 자료의 예제에서도 <code>cp.set()</code> 과 <code>showPoint()</code> 를 사용한다.
13	③	슈퍼 클래스의 <code>private</code> 멤버는 해당 클래스 내부에서만 직접 접근할 수 있다. 서브 클래스에서도 직접 접근할 수 없다.
14	④	<code>public</code> 멤버는 다른 모든 클래스에서 접근할 수 있다.
15	①	<code>protected</code> 멤버는 같은 패키지의 모든 클래스에서 접근할 수 있다. 또한 다른 패키지에 있는 서브 클래스에서도 접근할 수 있다.
16	②	다른 패키지에 있는 서브 클래스도 슈퍼 클래스의 <code>protected</code> 멤버에 접근할 수 있다.
17	①	default 멤버는 같은 패키지 내의 클래스에서 접근할 수 있다.
18	①	<code>weight</code> 는 <code>private</code> 이므로 Student에서 직접 접근할 수 없다. 자료의 예제에서는 <code>setWeight()</code> 를 이용해 간접적으로 접근한다.

19~22. 슈퍼 클래스와 서브 클래스의 생성자

문항	정답	해설
19	③	서브 클래스 객체가 생성될 때 슈퍼 클래스 생성자와 서브 클래스 생성자가 모두 실행된다.
20	②	실제 실행 순서는 슈퍼 클래스 생성자 → 서브 클래스 생성자이다.
21	②	<code>new B()</code> 를 실행하면 먼저 A의 생성자가 실행되어 <code>A</code> 를 출력하고 이후 B의 생성자가 <code>B</code> 를 출력한다. 따라서 <code>A B</code> 이다.
22	②	C 객체를 생성하면 상속 계층의 가장 위쪽 생성자부터 실행된다. 따라서 <code>A → B → C</code> 순서이다.

23~27. super()와 생성자 선택

문항	정답	해설
23	③	서브 클래스에서 슈퍼 클래스의 생성자를 명시적으로 선택하여 호출할 때 <code>super()</code> 를 사용한다.
24	②	서브 클래스 생성자에서 슈퍼 클래스 생성자를 선택하지 않으면 컴파일러가 슈퍼 클래스의 기본 생성자를 자동으로 선택한다.
25	③	슈퍼 클래스에 기본 생성자가 없는데 서브 클래스가 다른 생성자를 명시적으로 선택하지 않으면 호출할 기본 생성자가 없으므로 컴파일 오류가 발생한다.
26	②	<code>super(parameter)</code> 는 전달된 인자에 맞는 슈퍼 클래스의 생성자를 선택하여 호출한다.
27	②	<code>super()</code> 또는 <code>super(parameter)</code> 는 반드시 서브 클래스 생성자의 첫 번째 문장에 작성해야 한다.

28~30. 코드 이해

문항	정답	해설
28	③	<code>new B(5)</code> → B 생성자의 <code>super(5)</code> → <code>A(int x)</code> 가 먼저 실행되어 <code>A5</code> 출력 → 다시 B 생성자로 돌아와 <code>B5</code> 출력. 따라서 <code>A5 B5</code> 이다.
29	②	<code>super(x, y)</code> 는 두 개의 <code>int</code> 매개변수를 받는 슈퍼 클래스의 <code>Point(int x, int y)</code> 생성자를 호출한다. 자료의 <code>ColorPoint</code> 예제에서도 같은 방법을 사용한다.
30	②	<code>super(5, 6)</code> 으로 Point의 <code>x=5</code> , <code>y=6</code> 이 초기화되고 <code>color</code> 에는 <code>"blue"</code> 가 저장된다. <code>showColorPoint()</code> 가 먼저 <code>blue</code> 를 출력한 뒤 <code>showPoint()</code> 가 <code>(5,6)</code> 을 출력하므로 결과는 <code>blue(5,6)</code> 이다.